

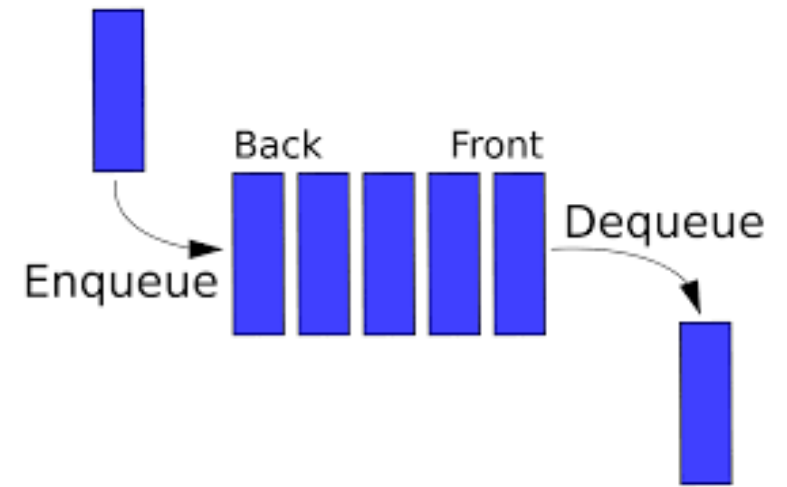
Structures de données linéaires:

ADT Queue

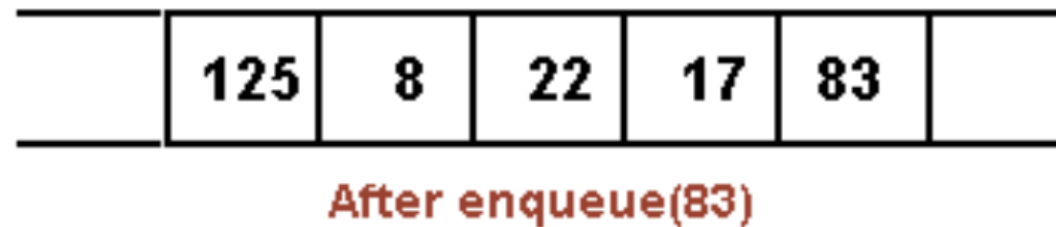
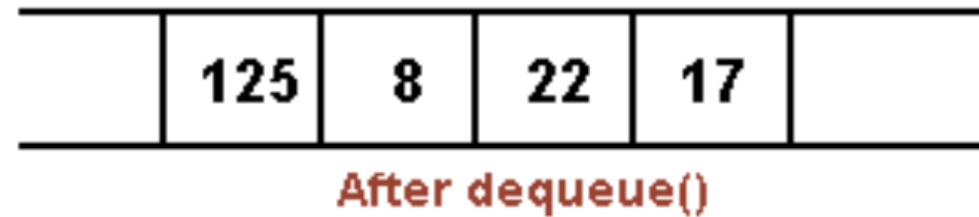
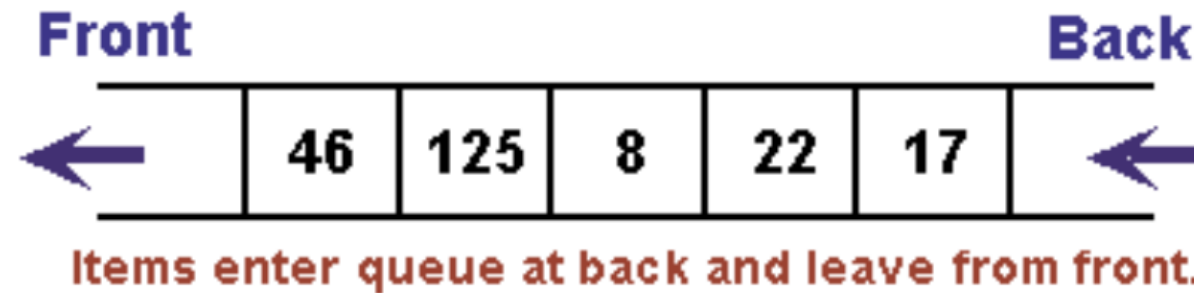
rouanehm@gmail.com

Notion de Queue (File)

- Une file est une structure linéaire permettant de stocker et de restaurer des données selon un ordre FIFO (First In, First Out ou « premier entré, premier sorti »)
- Liste dans laquelle seule l'insertion et la lecture se font à des extrémités opposées :
 - Les insertions (enfilements) se font à une extrémité appelée queue de la file.
 - Les suppressions (défilements) se font à l'autre extrémité appelée tête de la file.
- Synonymes : file de priorité, queue.



Manipulation d'une file



Opérations sur une file

Operation	Description
enqueue	Adds an element to the rear of the queue.
dequeue	Removes an element from the front of the queue.
first	Examines the element at the front of the queue.
isEmpty	Determines if the queue is empty.
size	Determines the number of elements on the queue.
toString	Returns a string representation of the queue.



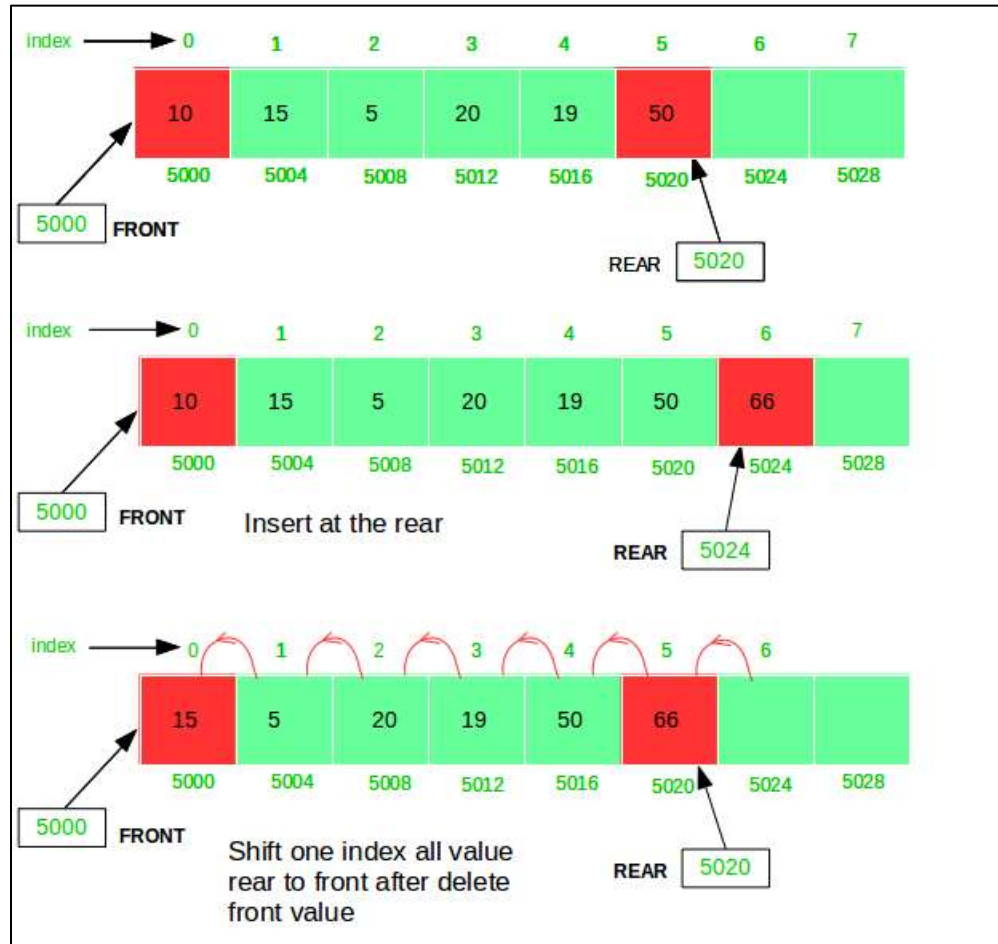
```
22 class Queue(object):
23
24     def __init__(self):
25         self.__items = deque()
26
27     def enqueue(self, item):
28         self.__items.append(item)
29
30     def dequeue(self):
31         self.__items.popleft()
32
33     def peek(self):
34         return self.__items[0]
35
36     def is_empty(self):
37         return len(self.__items) == 0
38
39     def size(self):
40         return len(self.__items)
41
42     def __str__(self):
```

Implémenter une file :

Les deux grandes alternatives

- Représentation contiguë :
 - Les éléments de la file sont rangés dans un tableau (simple ou circulaire)
 - Deux entiers représentent respectivement les positions de la tête et de la queue de la file
- Représentation chaînée :
 - Les éléments de la file sont chaînés entre eux
 - Un pointeur sur le premier élément désigne la file et représente la tête de cette file
 - Un pointeur sur le dernier élément représente la queue de file
 - Une file vide est représentée par le pointeur NULL

Implémenter une file : Tableau ordinaire



```
7 def __init__(self, capacity: int = 100):
8     self.__capacity = capacity
9     self.__front = 0
10    self.__rear = 0
11    self.__items = Queue.build_array(self.__capacity)
12
13    def enqueue(self, item):
14        if self.__capacity == self.__rear:
15            print('Queue is full.')
16            return
17        self.__items[self.__rear] = item
18        self.__rear += 1
19
20    def dequeue(self):
21        if self.is_empty():
22            print('Queue is empty')
23            return None
24        to_return = self.__items[self.__front]
25        self.shift_left(self.__front)
26        self.__rear -= 1
27        return to_return
```

Implémenter une file : Tableau circulaire

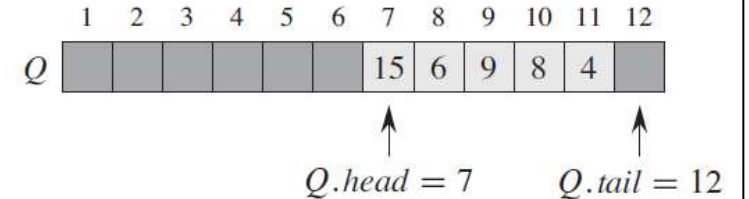
ENQUEUE(Q,X)

```
1  Q[Q.tail] = x
2  if Q.tail == Q.length
3      Q.tail = 1
4  else Q.tail = Q.tail + 1
```

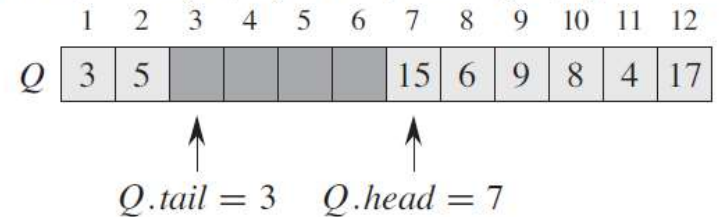
DEQUEUE(Q)

```
1  x = Q[Q.head]
2  if Q.head == Q.length
3      Q.head = 1
4  else Q.head = Q.head + 1
5  return x
```

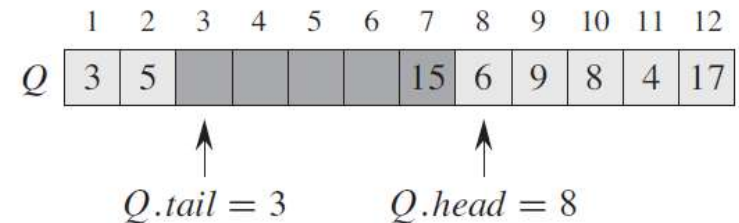
Etat initial :



ENQUEUE(Q, 17), ENQUEUE(Q, 3), ENQUEUE(Q, 5)

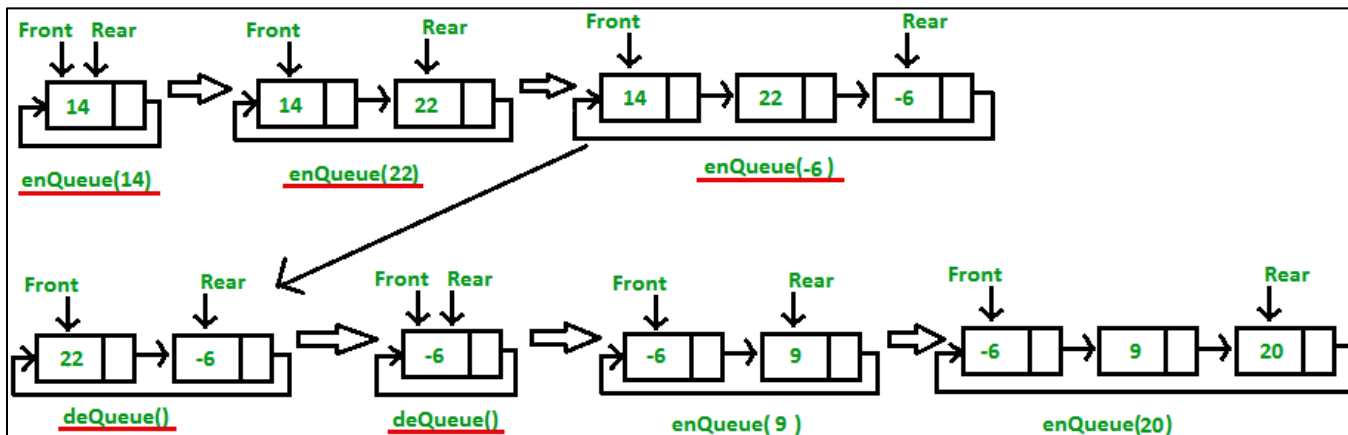


DEQUEUE(Q) → 15



Implémenter une file : Liste chaînée simple

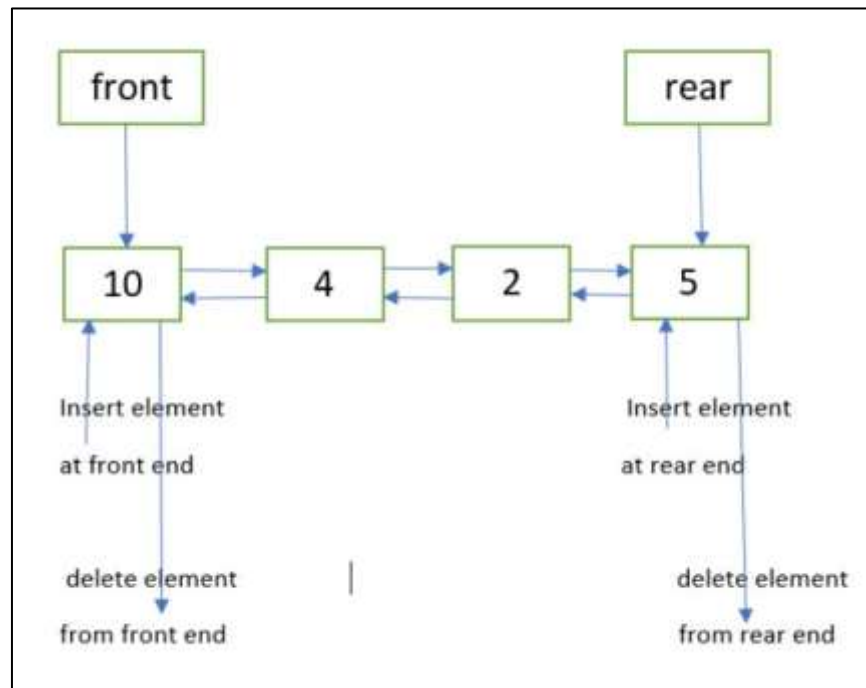
```
2 class Node(object):
3     def __init__(self, item: int, next_node=None):
4         self.item = item
5         self.next = next_node
6
7     def __repr__(self):
8         return repr(self.item)
9
10
11 # DoubleHeadedSinglyLinkedList (DHSL) Based Queue
12 class Queue(object):
13     def __init__(self):
14         self.__head = None
15         self.__tail = self.__head
16         self.__current_size = 0
```



```
32 def add_back(self, item):
33     new_node = Node(item, None)
34     self.__current_size += 1
35     if self.__head is None:
36         self.__head = new_node
37         self.__tail = new_node
38         return
39     self.__tail.next = new_node
40     self.__tail = new_node
41
42 def enqueue(self, item):
43     self.add_back(item)
```

```
59 def remove_front(self):
60     self.__current_size -= 1
61     if self.__head is None:
62         return None
63     to_return = self.__head.item
64     self.__head = self.__head.next
65     return to_return
66
67 def dequeue(self):
68     return self.remove_front()
69
70 def peek(self):
71     return self.__head.item
```

Implémenter une file : Liste doublement chaînée



<https://www.geeksforgeeks.org/implementation-deque-using-doubly-linked-list/>

```
36 def add_back(self, item):
37     new_node = Node(item, self.__tail, None)
38     if self.__head is None:
39         self.__head = new_node
40         self.__tail = new_node
41     return
42     self.__tail.next = new_node
43     self.__tail = new_node
44
45 def remove_front(self):
46     return self.remove_at(0)
```

```
48 def remove_at(self, index):
49     if not 0 <= index < self.length():
50         return IndexError('Provided index is out of bounds')
51     if index == 0:
52         self.__head = self.__head.next
53         if self.__head is not None:
54             self.__head.previous = None
55         else:
56             self.__tail = None
57     return
58     if index == self.length()-1:
59         self.__tail = self.__tail.previous
60         self.__tail.next = None
61     current = self.__head
62     rank = 0
63     while rank != index:
64         current = current.next
65         rank += 1
66     # remove current node
67     current.previous.next = current.next
68     current.next.previous = current.previous
69     return current.item
```

Patron adaptateur :

Utiliser une liste pour implémenter une file

- Deux alternatives :
 - Héritage :
 - La file hérite de la liste. Les méthodes de la liste sont protégées et donc accessible uniquement par la file.
 - Délégation :
 - le patron de conception Adapter
 - La file est un adaptateur
 - La liste est l'adaptée

```
1  from src.adt.list.SinglyLinkedList import SinglyLinkedList
2
3
4  class QueueUsingInheritance(object, SinglyLinkedList):
5
6      def enqueue(self, item):
7          self.add_back(item)
8
9      def dequeue(self):
10         self.remove_front()
11
12
13  class QueueUsingDelegation(object):
14
15      def __init__(self):
16         self.__queue = SinglyLinkedList()
17
18      def enqueue(self, item):
19         self.__queue.add_back(item)
20
21      def dequeue(self):
22         self.__queue.remove_front()
```

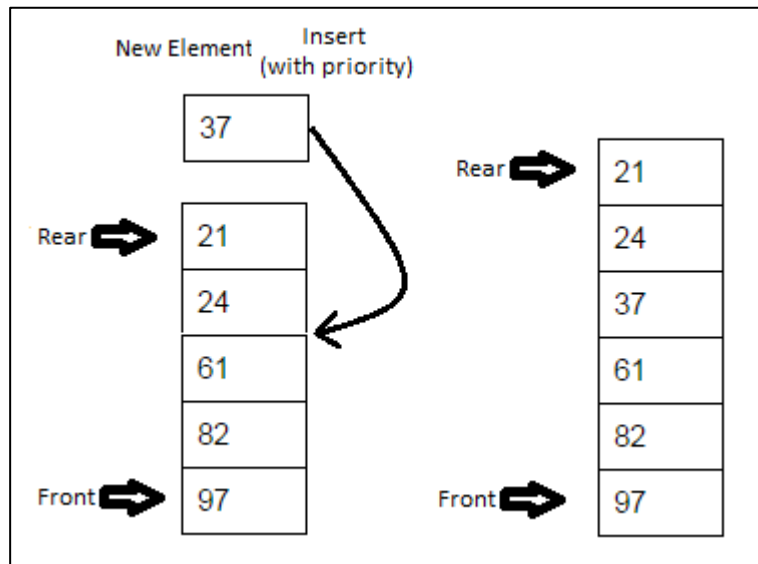
Complexité des opérations dans une file

Opérations	Tableau Simple	Tableau circulaire Avec deux indices	Liste chaine simple avec head et tail	Liste doublement chaînée circulaire avec une seule tête (head)
enqueue()	$O(1) \rightarrow O(n)$ Redimens.	$O(1) \rightarrow O(n)$ redimensionnement	$O(1) \rightarrow O(n)$	$O(1)$
dequeue()	$O(n)$	$O(1)$	$O(1)$	$O(1)$
front()/peek()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
length()	$O(1)$	$O(1)$	$O(1) \rightarrow O(n)$ Si pas de var size	$O(1) \rightarrow O(n)$ Si pas de var size

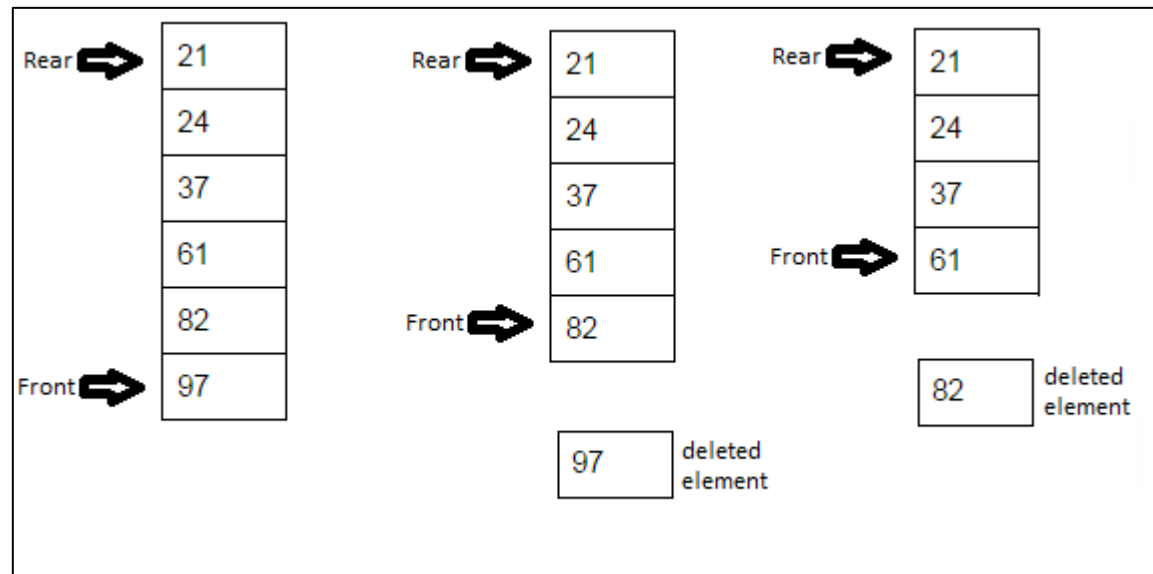
File à priorité

- Ensemble dynamique d'objets classés par ordre de priorité
 - Permet d'extraire un objet possédant la plus grande priorité
 - En pratique, on représente les priorités par les clés
 - Suppose un ordre total défini sur les clés

Insérer un élément



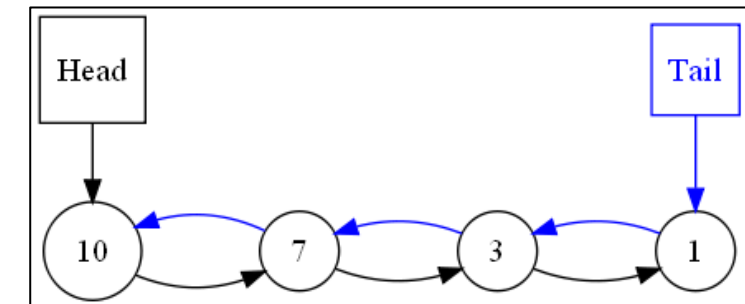
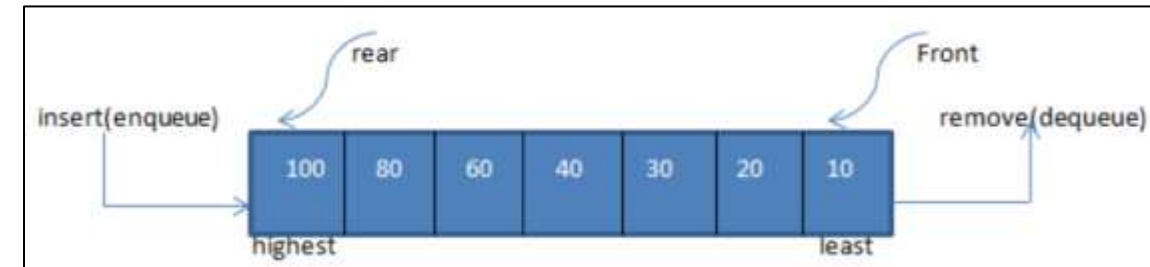
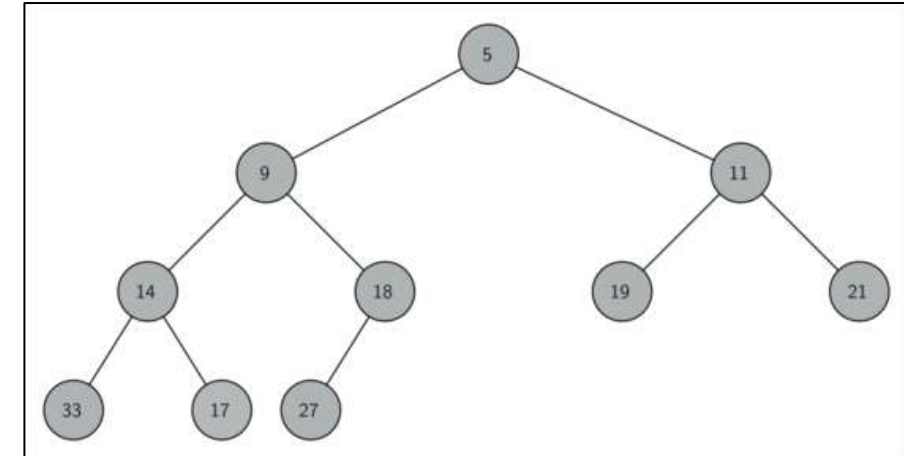
Supprimer un élément



Implémenter une file à priorité :

Le principe du Heap

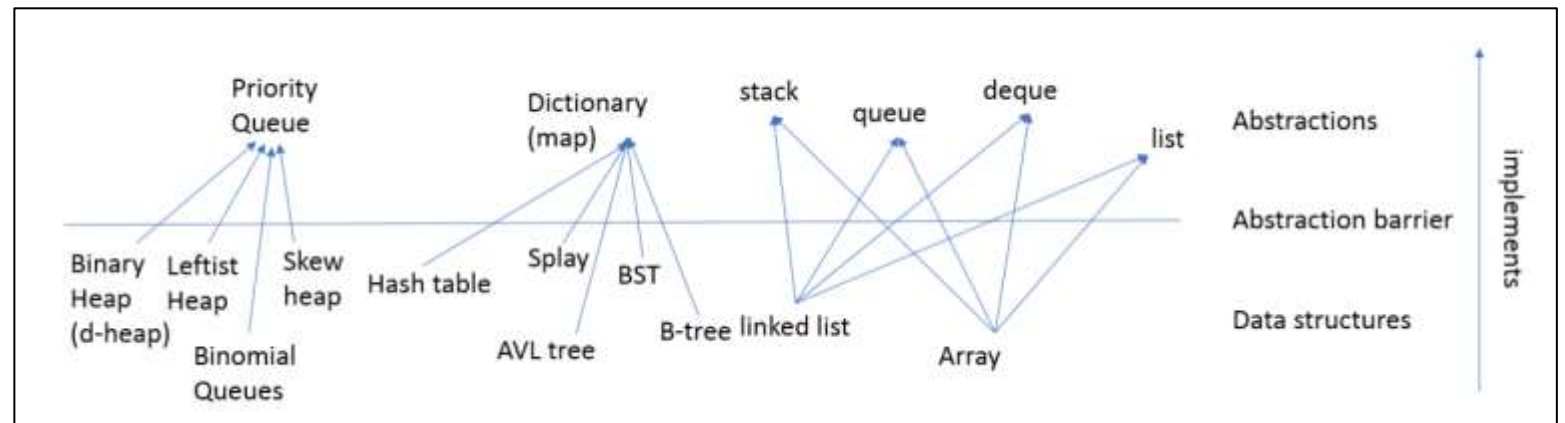
- La structure fondamentale d'une file à priorité est un arbre binaire avec la condition qu'un nœud est l'élément le plus prioritaire de tous ses descendants (un Heap).
- Trois alternatives :
 - Arbre binaire complet réel
 - Arbre binaire complet dans un tableau
 - Arbre binaire complet dans une liste chaînée
- **Note** : un arbre binaire est complet si tous les niveaux sont pleins, excepté le dernier niveau. Un niveau h peut avoir au max 2^h



Implémenter une file à priorité : ADT Heap

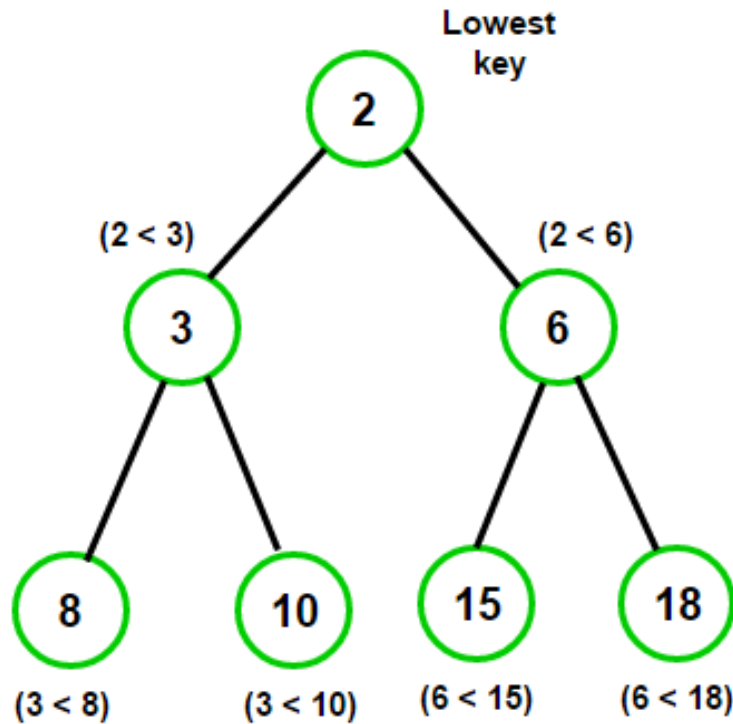
The basic operations we will implement for our binary heap are as follows:

- **BinaryHeap()** creates a new, empty, binary heap.
- **insert(k)** adds a new item to the heap.
- **find_min()** returns the item with the minimum key value, leaving item in the heap.
- **del_min()** returns the item with the minimum key value, removing the item from the heap.
- **is_empty()** returns true if the heap is empty, false otherwise.
- **size()** returns the number of items in the heap.
- **build_heap(list)** builds a new heap from a list of keys.

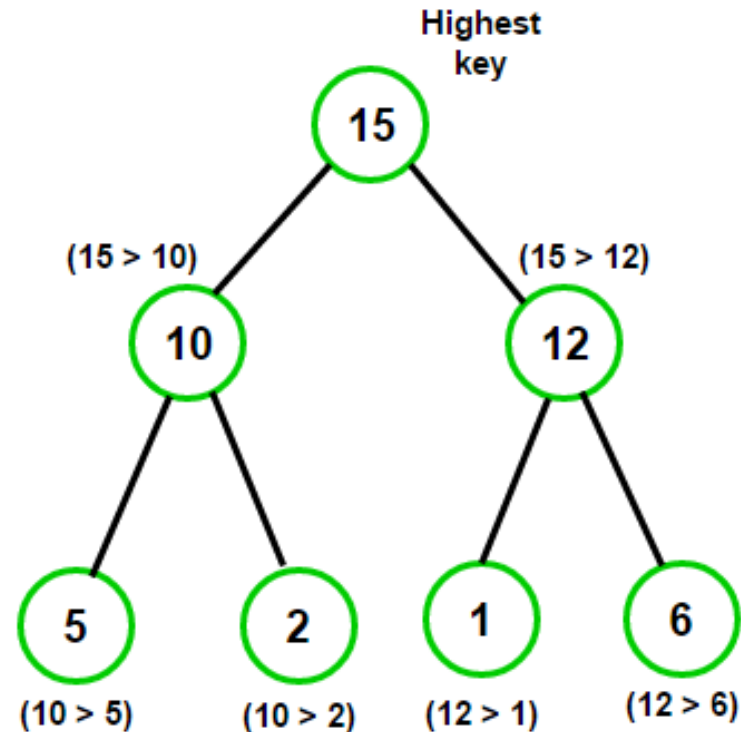


Implémenter une file à priorité :

Min Heap vs Max Heap



Min Heap
(Parent key is less than or equal to $\leq$ the child key)

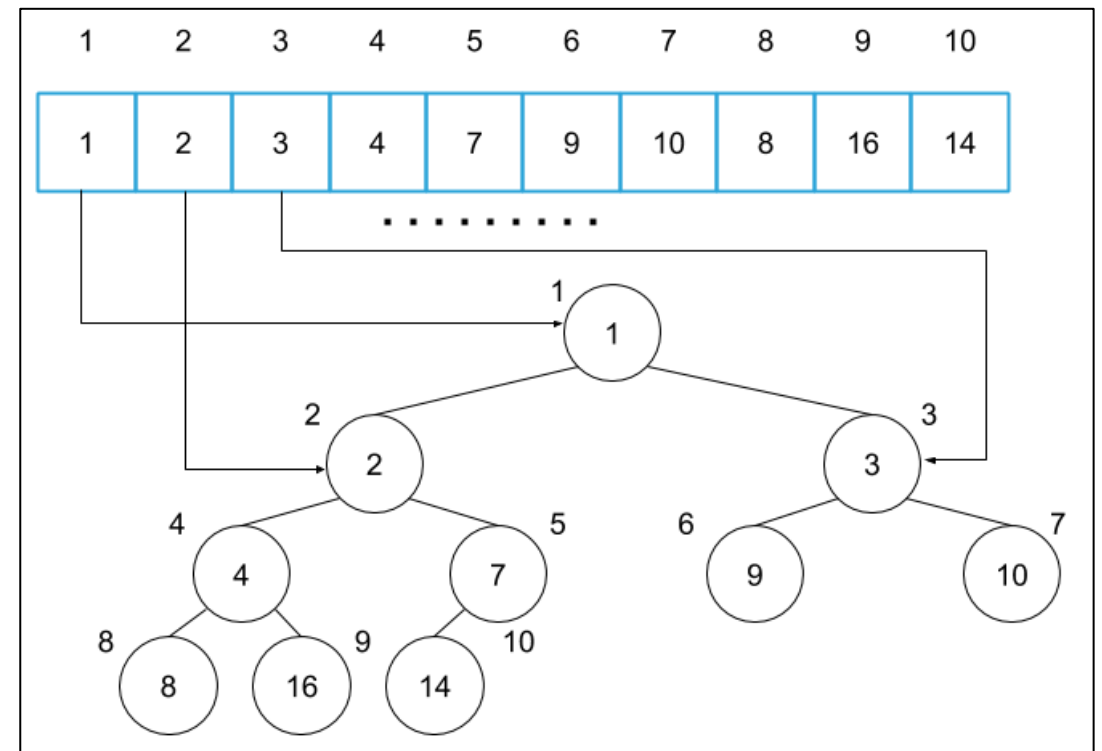


Max Heap
(Parent key is greater than or equal to $\geq$ the child key)

Implémenter une file à priorité :

Représenter un Heap avec un tableau

- Représentation typique
- Les éléments sont rangés dans un tableau avec les conditions ci-dessous :
- Pour un élément d'indice i , ses enfants sont stockés aux indices $2*i$ (gauche) et $2*i+1$ (droit)
- La racine est l'élément le plus prioritaire.

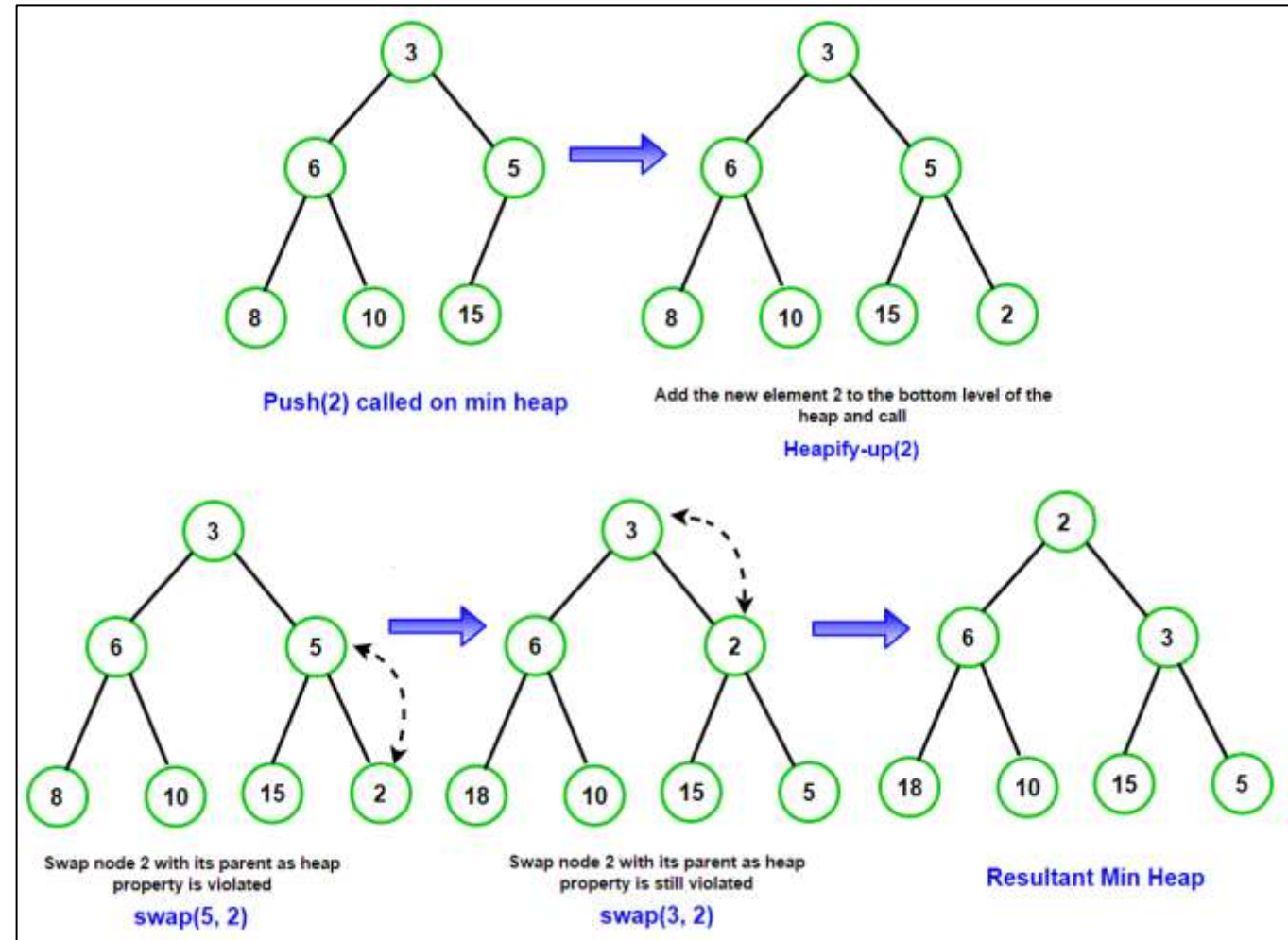


Implémenter un Heap avec un tableau :

Opération insert() et heapify_up(i)

- L'opération insert (ou push) permet d'insérer un nouveau item dans la file dont il faudra trouver la bonne position afin de respecter l'ordre dans le heap.
- L'opération insert ajoute le nouveau item à la fin du tableau et est toujours suivie de l'opération heapify_up(i) qui permet de rétablir l'ordre en comparant le nœud avec son parent de façon récursive en commençant par le nouvel item ajouté.

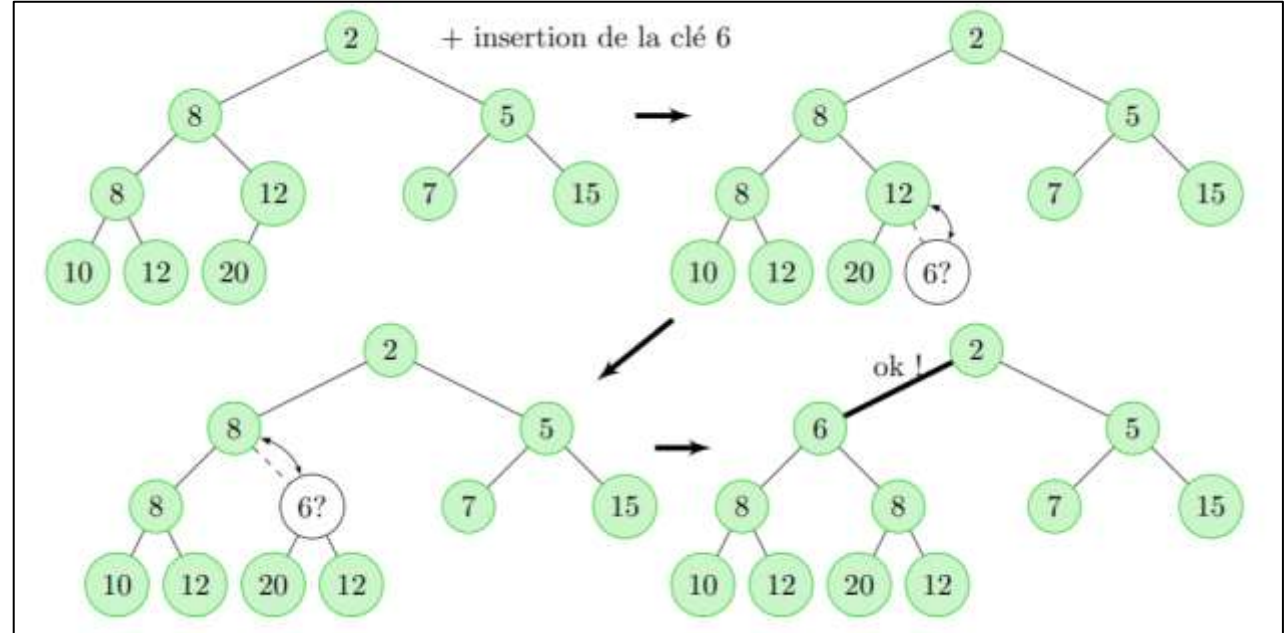
<https://www.techiedelight.com/introduction-priority-queues-using-binary-heaps/>



Implémenter un Heap avec un tableau :

Opération insert() et heapify_up(i)

```
2 class MinHeap(object):
3
4     def __init__(self):
5         self.__keys = [None]
6         self.current_size = 0
7
8     def insert(self, key):
9         self.__keys.append(key)
10        self.current_size += 1
11        self.heapify_up(self.current_size)
12
13    def heapify_up(self, index: int):
14        while index // 2 > 0:
15            if self.__keys[index] < self.__keys[index // 2]:
16                swap = self.__keys[index // 2]
17                self.__keys[index // 2] = self.__keys[index]
18                self.__keys[index] = swap
19                index = index // 2
20
21    def peek(self):
22        return self.__keys[1]
```

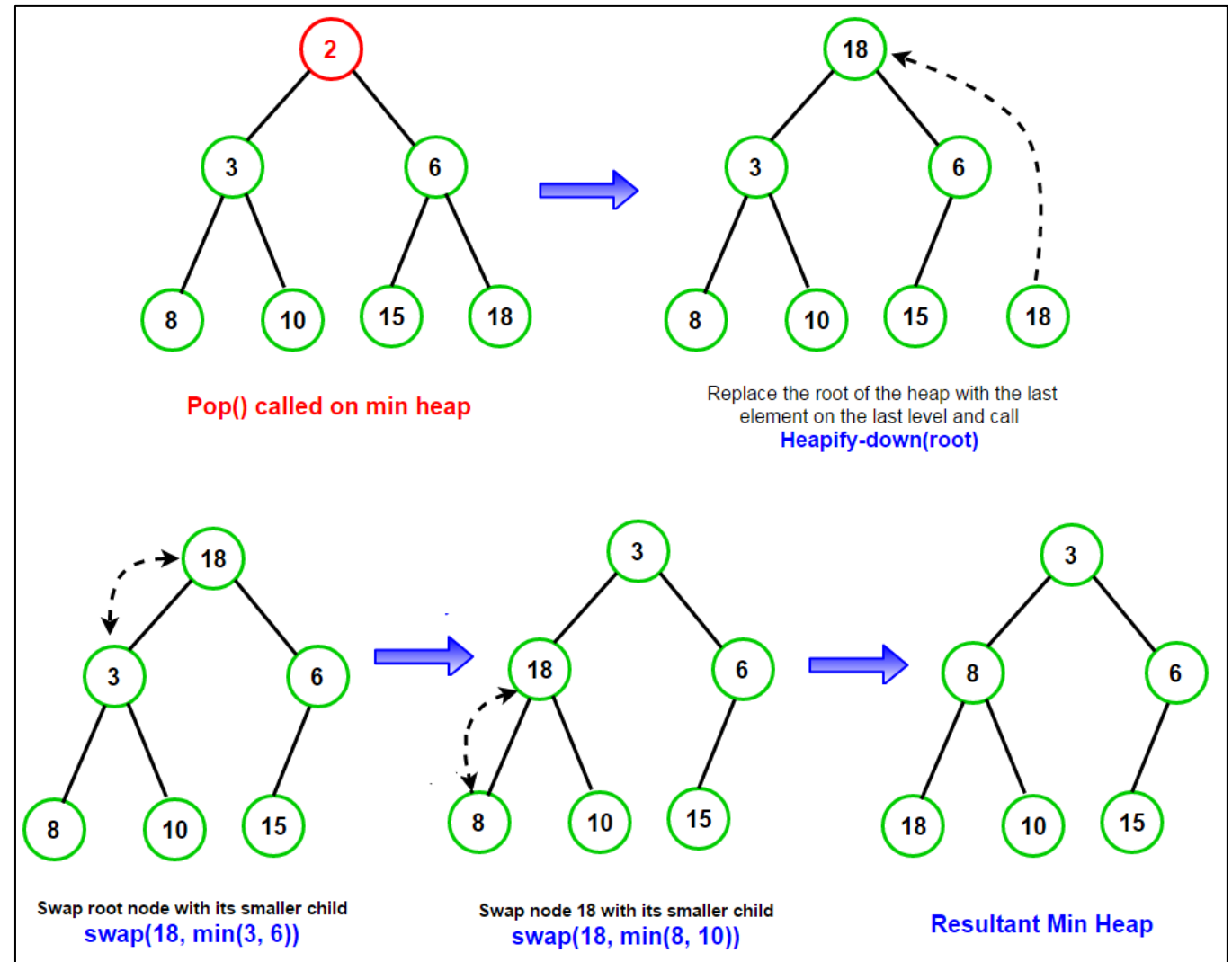


Implémenter un Heap avec un tableau :

Opération remove() et heapify_down(i)

- remove (ou pop) permet de retirer du heap (la file à priorité) l'élément le plus prioritaire, i.e, **la racine**.
- La racine est remplacée par le dernier éléments dans le tableau (le dernier niveau de l'arbre, le plus droite).
- Le remplacement de la racine provoque une incohérence dans le principe du heap et il faut le réarranger en invoquant l'opération heapify_down() qui remplace un nœud racine par l'un de ses fils de façon récursive jusqu'en bas de l'arbre.

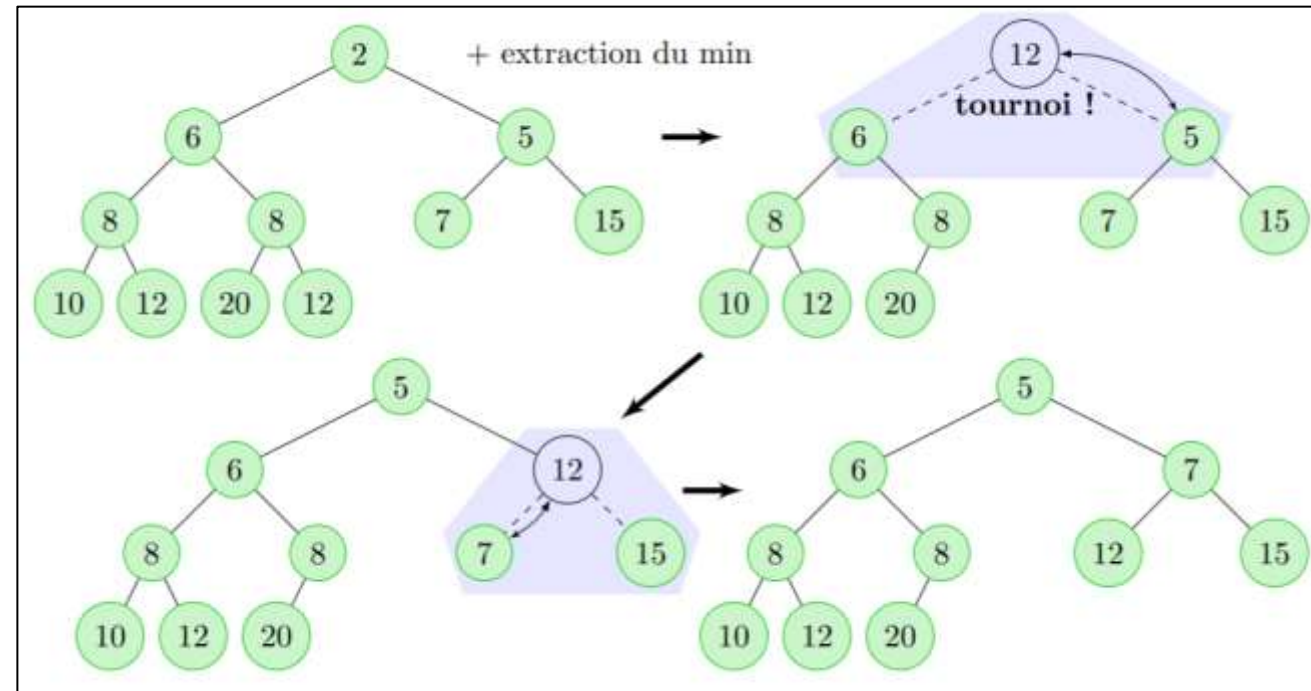
<https://www.techiedelight.com/introduction-priority-queues-using-binary-heaps/>



Implémenter un Heap avec un tableau :

Opération remove() et heapify_down(i)

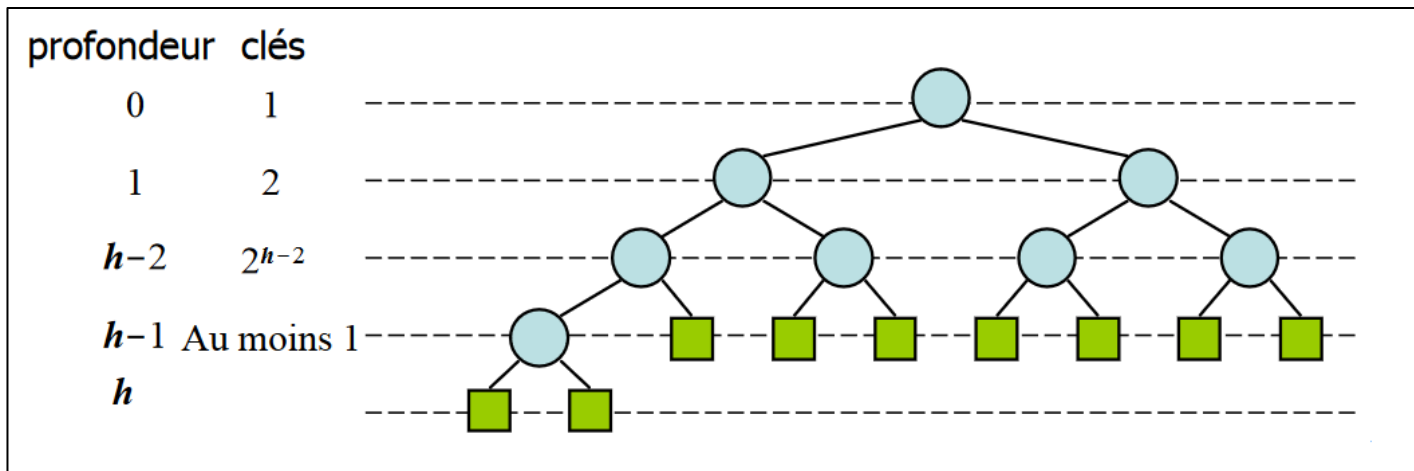
```
24 def remove(self):
25     to_return = self.__keys[1]
26     self.__keys[1] = self.__keys[self.current_size]
27     self.current_size -= 1
28     self.__keys.pop()
29     self.heapify_down(1)
30     return to_return
31
32 def heapify_down(self, index: int):
33     while (index * 2) <= self.current_size:
34         min_child = self.min_child(index)
35         if self.__keys[index] > self.__keys[min_child]:
36             swap = self.__keys[index]
37             self.__keys[index] = self.__keys[min_child]
38             self.__keys[min_child] = swap
39             index = min_child
40
41 def min_child(self, index):
42     if index * 2 + 1 > self.current_size:
43         return index * 2
44     else:
45         if self.__keys[index * 2] < self.__keys[index * 2 + 1]:
46             return index * 2
47         else:
48             return index * 2 + 1
```



Implémenter un Heap avec un tableau :

Ordre de grandeur de la hauteur d'un heap

- Un heap qui stocke n clés a une hauteur $h = O(\log_2 n)$
- Preuve:
 - Nombre total des nœuds dans un arbre binaire complet de hauteur h est comme suit:
 $n = 1 + 2 + 4 + \dots + 2^h$
 - Nous savons que $1 + 2 + 4 + \dots + 2^h = 2^{(h+1)} - 1$. Ainsi, $n = 2^{(h+1)} - 1$
 - Donc hauteur $h = \log_2(n + 1) - 1$
 - Par conséquent, h est de $O(\log n)$ en pire cas pour beaucoup de genres d'arbres de recherche équilibrés (AVL, rouge-noir, arbre 2-3-4, etc.)



Height	Nodes	Log calculation
0	1	$\log_2 1 = 0$
1	3	$\log_2 3 = 1$
2	7	$\log_2 7 = 2$
3	15	$\log_2 15 = 3$

Implémenter un Heap avec un tableau :

Complexité des opérations

- Les opérations prennent un temps proportionnel à la hauteur (nombre de niveau) de l'arbre.
- L'opération heapify réalise à chaque étape un certain nombre d'opérations en un temps constant.
- Ainsi, insert, remove sont tous les deux en $O(\log_2 n)$.

Algorithm	Worst case
Space	$O(n)$
Search	$O(n)$
Insert	$O(\log n)$
Delete	$O(\log n)$
Peek	$O(1)$

Applications des files à priorité

- Run multiple programs in the operating system
 - “critical” before “interactive” before “compute-intensive”
 - Maybe let users set priority level
- Treat hospital patients in order of severity (or triage)
- Select print jobs in order of decreasing length?
- Forward network packets in order of urgency
- Select most frequent symbols for data compression (cf. capstone project)
- Sort: insert all, then repeatedly deleteMin (HeapSort)
- Discrete event simulation

Références

- <https://algorithmtutor.com/Data-Structures/Basic/Queues/>
- <https://www.geeksforgeeks.org/circular-queue-set-1-introduction-array-implementation/>
- <http://algorithms.mrseliasclasses.org/queue-adt/>
- <https://jipe.developpez.com/articles/algo/file-priorite/#LI-A>
- <https://towardsdatascience.com/data-structure-heap-23d4c78a6962>
- <https://www.techiedelight.com/introduction-priority-queues-using-binary-heaps/>
- <https://runestone.academy/runestone/books/published/pythonds/Trees/BinaryHeapImplementation.html>
- <https://www.irif.fr/~francoisl/DIVERS/l3algo1819tasfiles.pdf>